

Deep Learning

Lecture 5: Energy-based models

Chris G. Willcocks

Durham University



1 High-level modelling trilemma

2 Manifolds

3 Energy-based models

- definition
- GANs as energy-based models
- clustering as an energy-based model
- softmax and softmin
- exact likelihood
- Boltzmann machines definition
- restricted and deep Boltzmann machines

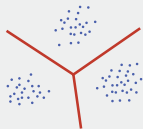
4 Probabilistic diffusion models

- approach and implementation
- flow matching
- diffusion-based anomaly detection
- VQ-GAN-EBM hybrids

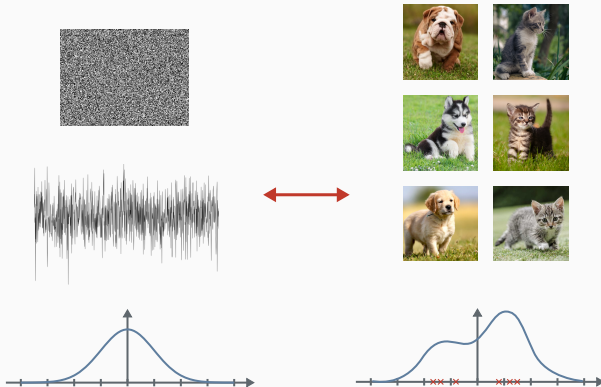
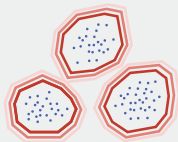
Generative models

- Learn a distribution over a dataset.
- Map between noise and data.
- Noise (prior) can be different dimension to data.

Discriminative modelling
(classification, regression)



Generative modelling
(sampling, density estimation)

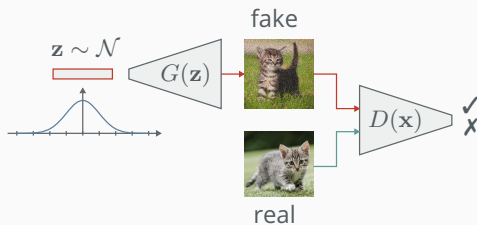


Definition: generative adversarial networks

A generative adversarial network (GAN) is a non-cooperative zero-sum game where two networks compete against each other [1].

One network $G(\mathbf{z})$ generates new samples, whereas D estimates the probability the sample was from the training data rather than G :

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] \\ + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

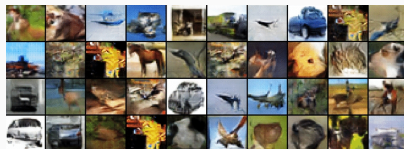
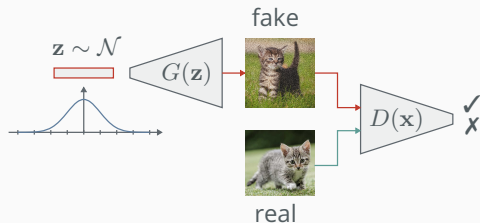


GAN properties

GANs benefit from differentiable data augmentation, but are otherwise have a variety of issues:

- Non-convergence
- Diminishing gradient
- Difficult to balance
- Mode collapse

[Link to Colab example](#) 

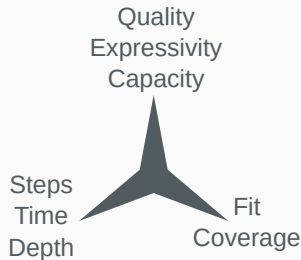


Question: the trilemma?

- They are fast (quick to sample)
- They have very poor coverage
- They have excellent quality

...2/3 ain't bad.

The generative modelling trilemma
(empirical observation)

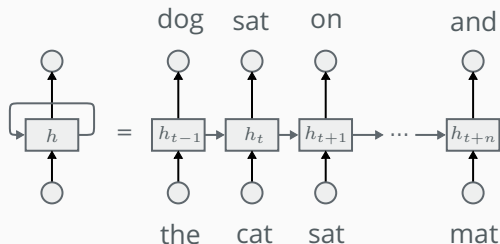


Definition: autoregressive (AR) generative models (e.g. chatGPT)

AR models maximise the likelihood of the training data (excellent mode coverage):

$$p_{\theta}(\mathbf{x}) = p_{\theta}(x_1, \dots, x_N) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

This is slow due to the sequential nature defined by the chain rule of probability.



Definition: diffusion probabilistic models (DPMs)

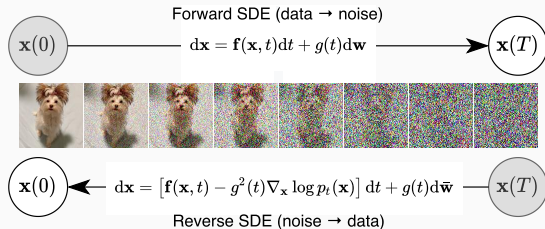
Instead of modelling a sequence of words, model a diffusion of noise in the data space. Define a forward (diffusion) equation:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t),$$

which can be reversed to sample the model:

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}).$$

This also requires a long iterative **transformation process**.

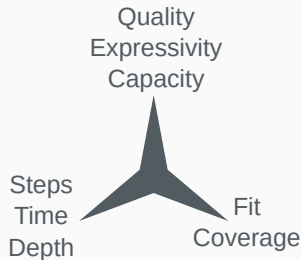


Question: where are these in the trilemma?

- They are slow (lots of iterations)
- They have excellent coverage
- They have excellent quality

...I'd do anything for 3/3...

The generative modelling trilemma
(empirical observation)



Definition: manifold

A manifold is a topological space that locally resembles Euclidean space

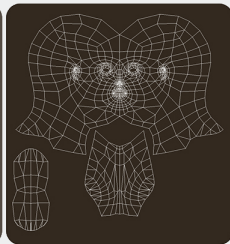
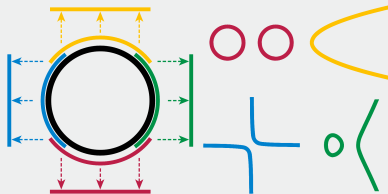
- topological manifold
- differentiable manifold
- Riemannian manifold

Definition: embedding

An embedding is a function ϕ that maps a manifold $\mathcal{M}$ to a new manifold $\mathcal{N}$ in an injective way that preserves its structure:

$$\phi : \mathcal{M} \rightarrow \mathcal{N}$$

Example: manifolds



Definition: energy-based models

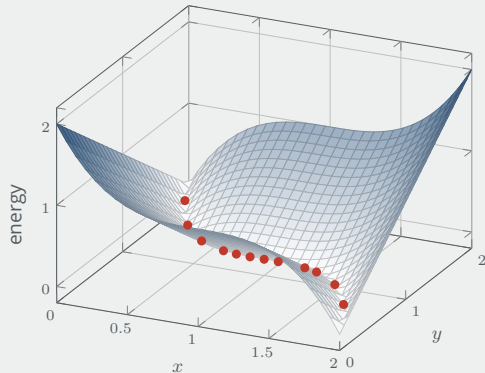
These are just any function that is happy when you input something that looks like data, and is not happy when you input something that doesn't look like data.

$$E(\mathbf{x}) = 0 \quad \checkmark$$

$$E(\tilde{\mathbf{x}}) > 0 \quad \times$$

This generic definition fits a large majority of machine learning models. For example $\mathcal{L}(E(\mathbf{x}), \mathbf{y})$ (a classifier)

Energy increases off manifold



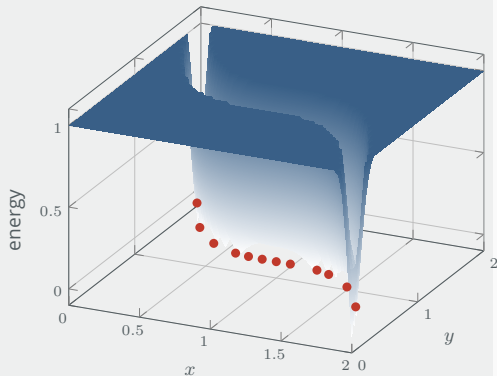
Definition: energy-based models

GANs are also energy models. The generator G generates samples off the manifold, then the discriminator D says these should be one everywhere, whereas it says real samples should be zero everywhere.

The generator also has to get good at sampling points on the data manifold. So it has to learn to generate points in the valley regions.

Is this smooth? What does a 1-Lipschitz discriminator do to the energy landscape?

GAN energy



Definition: clustering algorithm

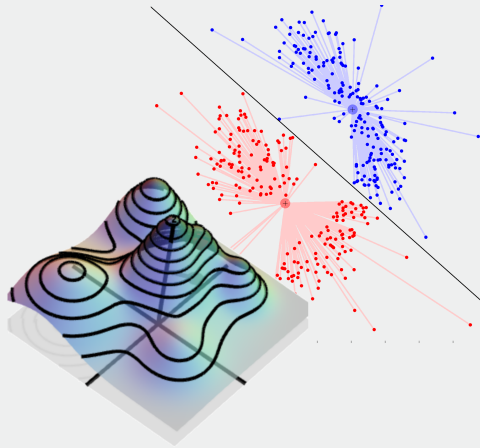
A cluster is a **connected-component** of a **level-set** of the **unknown PDF** over our data observations.

Traditionally:

- We don't know the PDF (the energy landscape)
- We don't necessarily know the level set
 - although 0.5 is appropriate for BCE
- This can be expensive (deep learning)

Click to watch a video that visually explains from the definition [!\[\]\(8942d28dc4da2a769efbb41dc37c5a1c_img.jpg\)](#)

Example: clustering by its definition





Definition: softmax and softmin

Softmax and softmin functions rescale elements to be in the range $[0, 1]$ and such that they sum to 1. So they create a probability mass function, e.g.:

$$\begin{bmatrix} 1.3 \\ 7.2 \\ 2.4 \\ 0.5 \\ 1.1 \end{bmatrix} \rightarrow \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \rightarrow \begin{bmatrix} 0.0027 \\ 0.9858 \\ 0.0081 \\ 0.0012 \\ 0.0022 \end{bmatrix}$$

Softmax functions are widely used (not just for EBM) where a distribution is needed, such as the last layer of a classifier.

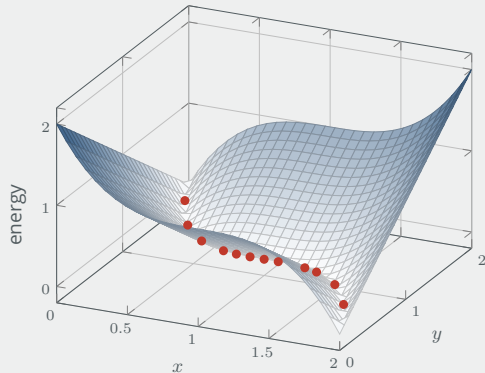
Challenges: energy-based models

EBMs are based on the observation that any probability density function $p(\mathbf{x})$ for $\mathbf{x} \in \mathbb{R}^n$ can be expressed as:

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\tilde{\mathbf{x}} \in \mathcal{X}} e^{-E(\tilde{\mathbf{x}})}},$$

where $E(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}$ is the energy function. However computation of the integral is intractable [2] for most models.

Energy increases off manifold



Definition: Boltzmann machine

Boltzmann machines [3] are one of the earliest neural networks for modeling binary data. They can associate the probability of the visible vectors $\mathbf{v}$ using finite summations:

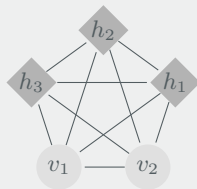
$$p_{\theta}(\mathbf{v}) = \frac{\sum_{\mathbf{h}} e^{-\beta E_{\theta}(\mathbf{v}, \mathbf{h})}}{\sum_{\tilde{\mathbf{v}}} \sum_{\mathbf{h}} e^{-\beta E_{\theta}(\tilde{\mathbf{v}}, \mathbf{h})}}$$

They are typically trained via negative log-likelihood through contrastive divergence, where the weights are updated:

$$\sum_{\mathbf{x} \in \mathcal{X}} \frac{\partial \ln p(\mathbf{x})}{\partial w_{i,j}} = \mathbb{E}_{p_d}[\mathbf{v}\mathbf{h}^T] - \mathbb{E}_{p_{\text{model}}}[\mathbf{v}\mathbf{h}^T]$$

Example: Boltzmann machine

They are an energy model which just have visible layers $v_1, v_2, \dots, v_n$ (inputs) and hidden layers $h_1, h_2, \dots, h_n$ (no outputs):



This example Boltzmann Machine has 2 visible units and 3 hidden units.



Definition: RBMs and DBMs

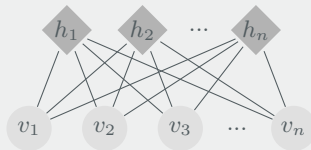
Restricted Boltzmann Machines (RBMs) and Deep Boltzmann Machines (DBMs) are Boltzmann machines with a more restricted (bipartite) graph structure [4]. DBMs have additional hidden layers.

That means that the visible units conditional on the hidden units become independent, which makes training these straightforward in practice.

[Link to Colab](#)  [Good YouTube talk](#) 

Example: RBM

RBMs have a restricted architecture so that there are no connections between hidden units:



DBMs are like the above, but with multiple hidden layers between.

Diffusion probabilistic modelling

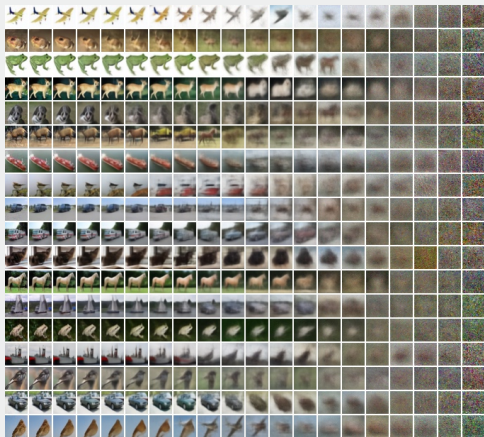
Diffusion probabilistic modelling approaches (such as DDPMs [5]) typically have a U-Net shaped architecture:

Data is gradually diffused in a forward process for T timesteps until it approximates the prior distribution.

The reverse process gradually removes noise, e.g. starting at $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ for T timesteps.

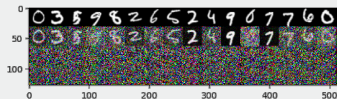
Like GANs, inject quality conditionals for better samples! (DALL-E 3, Stable Diffusion).

Example: CIFAR10 samples from [5]



DDPM implementation covered in the practicals

Predicting noise with U-Net



```
class DDPM(nn.Module):
    def __init__(self):
        super(DDPM, self).__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64, 3, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(in_channels=64, out_channels=3, 3, 2, 1)
        )

    # algorithm 1 in DDPM paper (simplified)
    def forward(self, x):
        ts = torch.randint(1, T+1, (x.shape[0],)).to(x.device)
        eps = torch.randn_like(x)
        x_t = nonlinear_blend(x, eps, ts)
        return F.mse_loss(eps, self.net(x_t))

    # algorithm 2 (simplified)
    def sample(self, n_sample, size):
        x_1 = torch.randn(n_sample, *size).to(device)
        for i in range(T, 0, -1):
            z = torch.randn(n_sample, *size).to(device) if i > 1 else 0
            eps = self.net(x_1)
            x_1 = schedule_func(x_1, eps, z, i)
        return x_1
```

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on

$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$$
- 6: **until** converged

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$

Definition: Flow matching

Flow matching [6] defines a simpler **ODE** between prior and data distribution:

$$\frac{dx_t}{dt} = v_\theta(x_t, t), \quad t \in [0, 1]$$

where $x_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (noise) and $x_1 \sim p_{\text{data}}$ (data) and x_t is e.g. a linear interpolant:

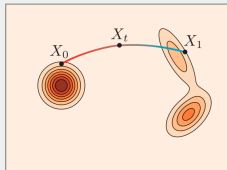
$$x_t = (1 - t)x_0 + tx_1$$

The model learns to predict velocity fields $(x_1 - x_0)$ by minimising:

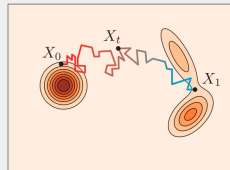
$$\mathcal{L} = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t) - (x_1 - x_0)\|^2]$$

Theory: Straight (optimal transport) paths in the transition space $\Rightarrow$ simpler and faster.

Comparison: straight vs. curved paths



Flow



Diffusion

Flow matching learns direct paths through the probability landscape, while DDPMs follow noisy diffusion trajectories.

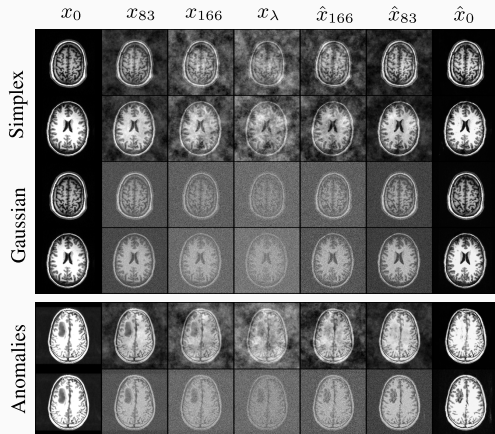
[Link to Colab example](#) 

Diffusion-based anomaly detection

Like GANs, diffusion-based models work well for anomalies (great for small datasets).

- Do a partial diffusion
- Train only on healthy/normal data
- Abnormal denoising will only know how to make the data look normal
- Any error = surprise = anomalies

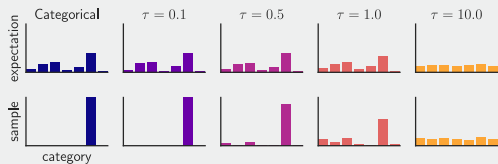
Our paper, AnoDDPM [7] (CVPR NTIRE), uses simplex noise to capture multi-scale anomalies. See also UNIT-DDPM [8] (unpaired translation).



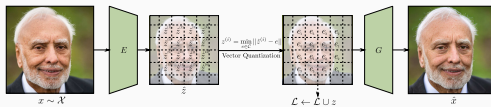
[Link to project page](#) 

Vector quantization

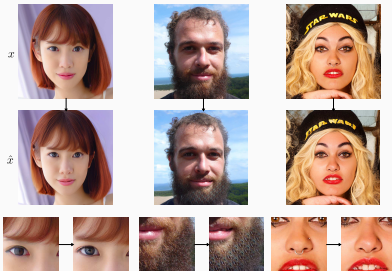
Imposing a discrete prior on the latents can be achieved with either variational or adversarial (non-blurry) approaches.



The Gumbel-Softmax distribution interpolates between discrete one-hot-encoded categorical distributions and continuous categorical densities.



Above: vector quantisation. **Below:** shift mode collapse to perceptually unimportant parts of the signal.







- [1] Ian Goodfellow et al. **“Generative adversarial nets”**. In: Advances in neural information processing systems. 2014, pp. 2672–2680.
- [2] Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, and F Huang. **“A tutorial on energy-based learning”**. In: Predicting structured data 1.0 (2006).
- [3] Geoffrey E Hinton and Terrence J Sejnowski. **“Optimal perceptual inference”**. In: Proceedings of the IEEE conference on Computer Vision and Pattern Recognition. Vol. 448. Citeseer. 1983.
- [4] Geoffrey E Hinton. **“Training products of experts by minimizing contrastive divergence”**. In: Neural computation 14.8 (2002), pp. 1771–1800.
- [5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. **“Denoising diffusion probabilistic models”**. In: arXiv preprint arXiv:2006.11239 (2020).
- [6] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. **“Flow Matching for Generative Modeling”**. In: The Eleventh International Conference on Learning Representations. 2023. URL: <https://openreview.net/forum?id=PqvMRDCJT9t>.



- [7] Julian Wyatt, Adam Leach, Sebastian M Schmon, and Chris G Willcocks. **“AnoDDPM: Anomaly Detection With Denoising Diffusion Probabilistic Models Using Simplex Noise”**. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022, pp. 650–656.
- [8] Hiroshi Sasaki, Chris G Willcocks, and Toby P Breckon. **“Unit-ddpm: Unpaired image translation with denoising diffusion probabilistic models”**. In: arXiv preprint arXiv:2104.05358 (2021).
- [9] Alex F McKinney and Chris G Willcocks. **“Megapixel Image Generation with Step-Unrolled Denoising Autoencoders”**. In: arXiv preprint arXiv:2206.12351 (2022).
- [10] Sam Bond-Taylor, Peter Hessey, Hiroshi Sasaki, Toby P Breckon, and Chris G Willcocks. **“Unleashing Transformers: Parallel Token Prediction with Discrete Absorbing Diffusion for Fast High-Resolution Image Generation from Vector-Quantized Codes”**. In: European Conference on Computer Vision (ECCV) (2022). DOI: 10.1007/978-3-031-20050-2_11.